SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	**CSA1206**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	**THIRUMALAPUDI PRANAVI**	**Reg.No.:**	**192411222**
Department:	**BE.CSE**	**Date:**	**22-8-2026**

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
   A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck
   A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing
   An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device
   An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console
   A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

***Code of Conduct**:*

*I **THIRUMALAPUDI PRANAVI (192411222**)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:T.Pranavi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Smartphone Performance Optimization

Introduction

Modern smartphones run multiple applications simultaneously. When a user switches from one application to another, the processor must quickly access the instructions and data of that application. If the required data is not available in the faster cache memories, the processor has to access DRAM, which takes more time. This can cause **application switching lag, increased latency and reduced performance**.

To solve this problem, a properly designed **memory hierarchy** using L1, L2, L3 cache and DRAM is required.

1. L1 Cache

L1 (Level 1) cache is the fastest and smallest memory in the hierarchy.

- It is located very close to the CPU core.
- It stores frequently used instructions and data.
- It has very low access latency.
- It is usually divided into **instruction cache and data cache**.
- Since it is small, only the most frequently required information can be stored.
- A high L1 cache hit rate allows the CPU to execute instructions without waiting for slower memory.

Role in smartphone performance:

During app switching, frequently used instructions and data can be obtained directly from L1 cache, reducing latency.

2. L2 Cache

L2 cache is larger than L1 but slightly slower.

- It acts as a secondary cache.
- It stores data that is not available in L1.
- It has greater capacity than L1.
- It reduces the number of accesses to L3 cache and DRAM.
- Each CPU core may have its own L2 cache depending on the processor architecture.

Role in smartphone performance:

If application data is missing from L1, L2 can provide the required data much faster than DRAM.

3. L3 Cache

L3 cache is larger than L1 and L2 but slower than them.

- It generally acts as a shared cache among multiple CPU cores.
- It stores frequently accessed data and instructions.
- It reduces communication with DRAM.
- It is useful when several CPU cores need access to common data.
- A larger and efficient L3 cache can improve overall system throughput.

Role in smartphone performance:

When an application is switched, its required data may be available in L3. This avoids a slower DRAM access and reduces application startup or switching time.

4. DRAM

DRAM (Dynamic Random Access Memory) is the main memory of the smartphone.

- It has much greater capacity than cache.
- Running applications, operating-system processes and their data are stored in DRAM.
- It is slower than L1, L2 and L3 cache.
- Modern smartphones use high-speed LPDDR memory.
- DRAM provides the large capacity required for multitasking.

Role in smartphone performance:

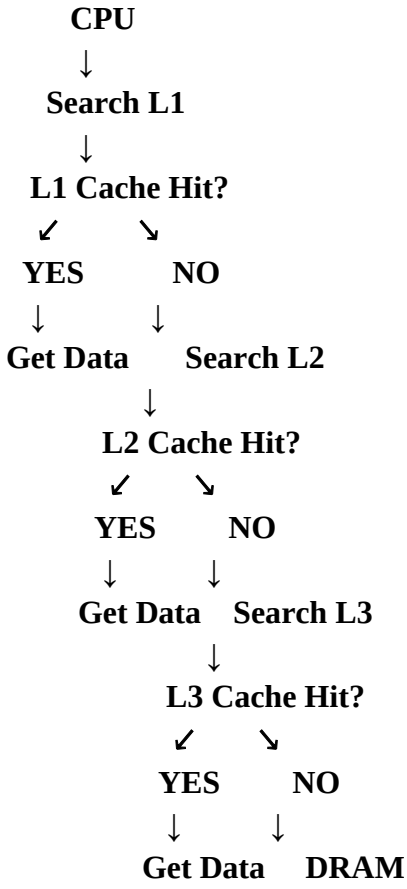
If the required application data is not available in any cache, the CPU has to access DRAM. Frequent DRAM accesses increase latency and power consumption.

5. Comparison of Memory Levels

Feature	L1 Cache	L2 Cache	L3 Cache	DRAM
Speed	Fastest	Very fast	Fast	Slower
Capacity	Very small	Small/medium	Larger	Very large
Latency	Very low	Low	Higher than L1/L2	High
Location	Closest to CPU	Near CPU	Shared/near CPU	Main memory
Cost per bit	Highest	High	Moderate	Lower
Main purpose	Immediate data	Secondary cache	Shared cache	Application storage

6. Memory Access Process

The CPU does not directly access DRAM every time. It searches the memory hierarchy from the fastest level to the slower levels.



A **cache hit** provides fast access, while a **cache miss** causes the processor to search the next memory level.

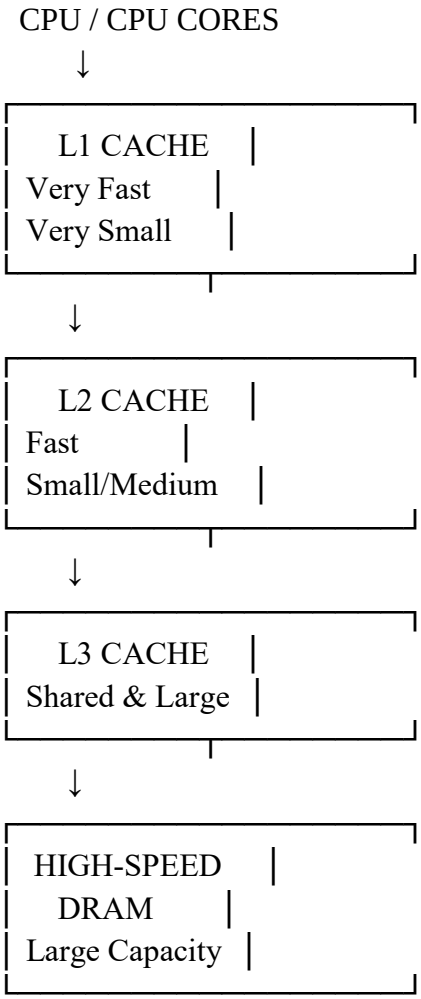
7. Cause of Smartphone Lag

The main causes of lag during application switching can include:

1. Low cache hit rate.
2. Frequent L1/L2/L3 cache misses.
3. Insufficient DRAM capacity.
4. Low DRAM bandwidth.
5. Large applications requiring more memory.
6. Too many background applications.
7. Poor cache replacement policies.
8. Lack of effective data prefetching.
9. Repeated loading of application data from slower memory.

8. Proposed Hierarchical Memory Redesign

The proposed memory architecture is:



9. Improvements Proposed

A. Optimize L1 Cache

Increase the efficiency of L1 cache so that frequently accessed application instructions and data remain close to the CPU.

B. Optimize L2 Cache

Provide sufficient L2 capacity to reduce the number of requests reaching L3.

C. Increase L3 Cache

A larger L3 cache can store more shared application data and reduce DRAM accesses.

D. Use High-Speed DRAM

Use high-bandwidth **LPDDR memory** to increase data-transfer speed between CPU and main memory.

E. Cache Prefetching

The processor can predict which data will be needed next and load it into cache in advance.

Example:

Application data needed soon

↓

Prefetch data

↓

Cache

↓

CPU accesses quickly

F. Improve Cache Replacement

Use efficient cache replacement techniques so that useful data remains in the cache instead of being removed unnecessarily.

G. Reduce DRAM Accesses

The main objective should be to increase cache hit rates and reduce unnecessary DRAM accesses.

H. Improve Multitasking

Sufficient DRAM capacity allows more applications to remain in memory, reducing the need to reload applications when switching between them.

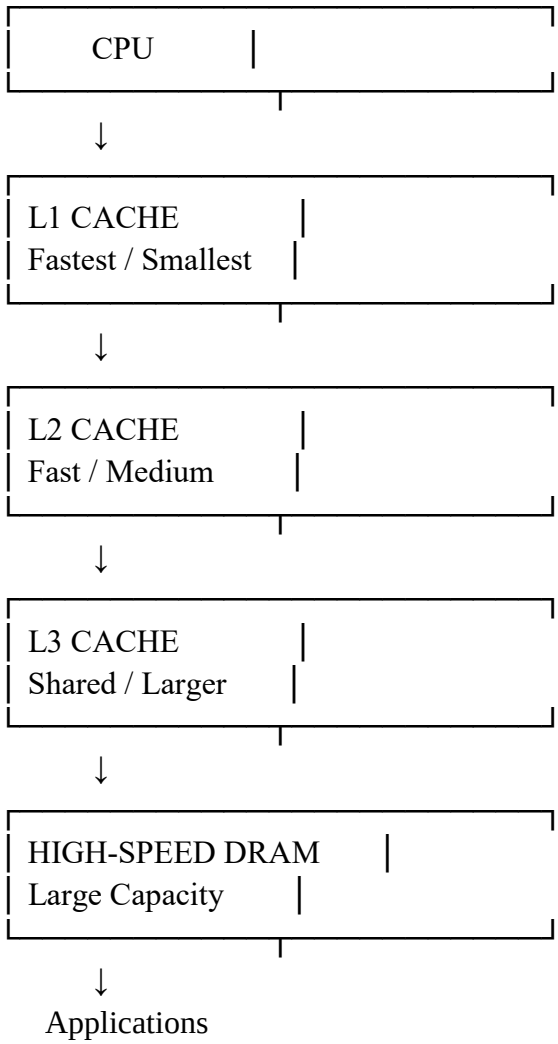
10. Expected Benefits

The proposed redesign provides:

- **Reduced memory latency**
- **Faster application switching**
- **Higher CPU utilization**
- **Improved memory bandwidth**
- **Better multitasking**
- **Reduced application reloads**
- **Improved smartphone responsiveness**
- **Lower performance loss caused by cache misses**

11. Overall Hierarchy

You can draw this final diagram in your assessment:



↑ Speed increases
↓ Capacity increases

Conclusion

A smartphone's performance depends heavily on the efficiency of its memory hierarchy. **L1 provides the lowest latency, L2 provides additional fast storage, L3 reduces accesses to main memory, and DRAM provides large-capacity working memory.** Therefore, the manufacturer should use optimized L1 and L2 caches, a sufficiently large shared L3 cache, high-bandwidth DRAM, cache prefetching and efficient cache-management techniques. This redesign will **reduce latency, improve throughput and provide smooth application switching**.

2. Cloud Server Memory Bottleneck

1. Introduction

Cloud servers handle a large number of database queries simultaneously. When users request information from a database, the CPU needs to access instructions and data from memory. If the required data is not available in fast cache memory, the system may access DRAM or even secondary storage.

This can result in **high latency, reduced throughput and poor database performance**. Two important techniques used in memory management are **cache hierarchy** and **virtual memory paging**.

2. Cache Hierarchy

Cache hierarchy consists of multiple levels of fast memory placed between the CPU and main memory.

The main levels are:

- **L1 Cache**
- **L2 Cache**
- **L3 Cache**
- **DRAM**

L1 Cache

- L1 is the smallest and fastest cache.
- It is located closest to the CPU.
- It stores frequently accessed instructions and data.
- It provides very low access latency.

L2 Cache

- L2 is larger than L1.
- It is slightly slower than L1.
- It stores data that is not available in L1.
- It reduces accesses to slower memory.

L3 Cache

- L3 is larger than L1 and L2.
- It is generally shared between CPU cores.
- It stores frequently accessed shared data.
- It reduces DRAM accesses.

DRAM

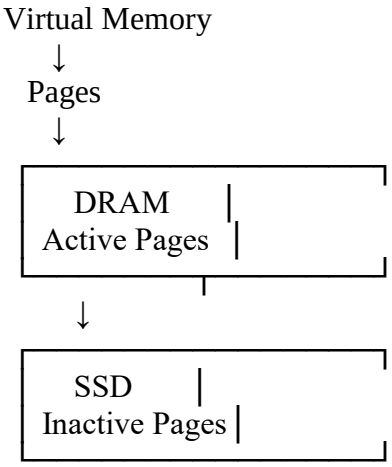
- DRAM acts as the main memory.
- It has much larger capacity than cache.

- It is slower than CPU caches.
- Running applications and database-related working data can reside in DRAM.

3. Virtual Memory Paging

Virtual memory allows a system to use secondary storage as an extension of main memory.

The operating system divides memory into fixed-size **pages**.



When a required page is present in DRAM, it can be accessed normally.

If the page is not present in DRAM, a **page fault** occurs. The operating system must retrieve the page from secondary storage.

This is much slower than accessing DRAM.

4. Cache Hierarchy vs Virtual Memory Paging

Feature	Cache Hierarchy	Virtual Memory Paging
Main purpose	Reduce CPU memory-access latency	Extend available memory
Managed by	Hardware + CPU	Operating system + hardware
Main technologies	SRAM cache + DRAM	DRAM + SSD
Speed	Very high	Comparatively slower
Unit	Cache line	Page
Failure/miss	Cache miss	Page fault
Main problem	Cache miss penalty	Very high page-fault latency
Best use	Frequently accessed data	Inactive/less frequently used data

5. Difference in Working

Cache Hierarchy

CPU
↓
L1
↓
L2
↓
L3
↓
DRAM

The CPU searches from the fastest cache to slower memory.

If data is found in cache, access is very fast.

Virtual Memory

CPU
↓
Virtual Address
↓
Page Table
↓
DRAM
↓
If page absent
↓
Page Fault
↓
SSD
↓
Page loaded into DRAM

A page fault causes significant delay because SSD access is much slower than cache or DRAM access.

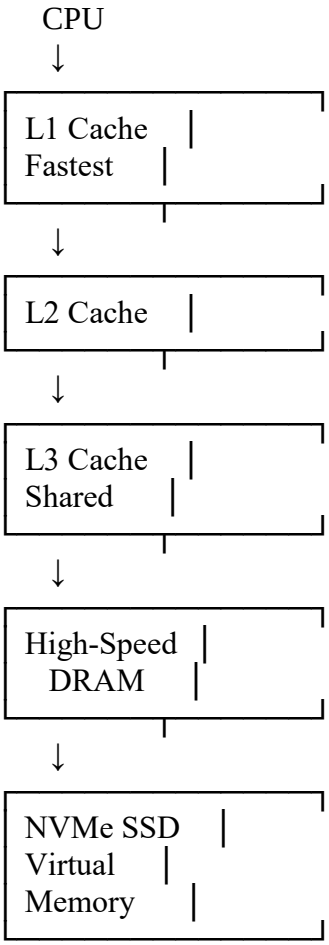
6. Why Database Queries Become Slow

High database-query latency can occur because of:

1. Frequent cache misses.
2. Insufficient CPU cache capacity.
3. Insufficient DRAM.
4. Low DRAM bandwidth.
5. Large database working sets.
6. Poor data locality.
7. Excessive virtual-memory paging.
8. Frequent page faults.
9. Slow secondary storage.
10. Inefficient database queries or indexing.

7. Recommended Memory Hierarchy

For a cloud database server, the recommended hierarchy is:



8. Recommended Improvements

1. Increase DRAM Capacity

Cloud servers should have sufficient DRAM to keep the active database working set in memory.

Benefit:
Reduces the need for virtual-memory paging.

2. Use High-Speed DRAM

Use modern high-bandwidth server memory such as **DDR5-class DRAM**.

Benefit:

- Higher memory bandwidth
- Faster data transfer
- Better performance for multiple database queries

3. Optimize L3 Cache

A sufficiently large L3 cache can store frequently accessed data and reduce DRAM accesses.

Benefit:
Improves CPU-side database processing.

4. Use NVMe SSD

If paging or storage access is unavoidable, high-performance **NVMe SSDs** are preferable to slower storage technologies.

Benefit:
Reduces the time required for storage I/O.

5. Reduce Page Faults

The operating system should keep frequently accessed pages in DRAM.

Benefit:
Reduces expensive transfers between DRAM and SSD.

6. Database Caching

Frequently accessed database records can be cached in memory.

For example:

Frequently Used Data
↓
Memory Cache
↓
Fast Database Response

This prevents repeated access to slower storage.

7. Optimize Database Queries

Database indexing and efficient query execution reduce unnecessary memory and storage operations.

For example:

Without Index
→ Search large amount of data
→ More memory access
→ Higher latency

With Index
→ Locate required record quickly
→ Fewer memory accesses

→ Lower latency

8. Use Memory Locality

Frequently accessed data should be placed close to the CPU whenever possible.

This improves:

- Cache hit rate
- CPU utilization
- Query response time
- Overall throughput

9. Cache Miss vs Page Fault

This is an important point to write in the assessment.

Cache Miss

A **cache miss** occurs when the CPU cannot find required data in the current cache level.

CPU → L1 Miss → L2 → L3 → DRAM

The penalty is relatively small compared with accessing storage.

Page Fault

A **page fault** occurs when the required virtual-memory page is not present in DRAM.

CPU → DRAM
↓
Page Fault
↓
NVMe SSD
↓
Load Page → DRAM

Page faults can cause very high latency.

10. Expected Benefits

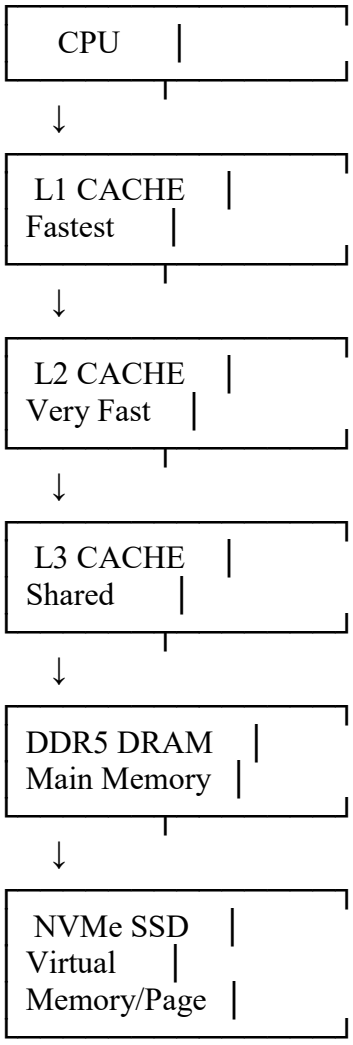
The proposed improvements provide:

- Reduced database-query latency
- Fewer cache misses
- Fewer page faults
- Higher memory bandwidth
- Faster database response
- Better CPU utilization
- Higher cloud-server throughput

- Better performance for multiple users
- Reduced dependency on slow storage

11. Final Memory Hierarchy

Draw this diagram in your assessment:



↑ Higher Speed
↓ Higher Capacity

Conclusion

Cache hierarchy and virtual memory paging solve different problems. **Cache hierarchy reduces CPU memory-access latency**, while **virtual memory paging allows the system to handle workloads larger than available DRAM**. For a cloud database server, the best solution is to use **large and efficient L1/L2/L3 caches, sufficient high-speed DDR5 DRAM and fast NVMe SSDs**, while minimizing page faults through proper memory management. This will reduce query latency and improve overall cloud-server throughput.

3. Autonomous Vehicle Data Processing

1. Introduction

An autonomous vehicle uses different sensors such as **cameras, LiDAR, radar, ultrasonic sensors and GPS** to continuously collect large amounts of data. This data must be processed in real time for functions such as object detection, lane detection, obstacle avoidance and navigation.

Therefore, the memory system must provide **very low latency, high bandwidth, sufficient capacity and low power consumption**.

Three important memory technologies that can be considered are **SRAM, DRAM and MRAM**.

2. SRAM

SRAM (Static Random Access Memory) is a very fast semiconductor memory.

Features

- Very high speed.
- Very low access latency.
- Does not require refresh operations.
- Volatile memory.
- Requires more area per bit than DRAM.
- More expensive than DRAM.
- Usually used for CPU and accelerator caches.

Role in Autonomous Vehicles

SRAM can store:

- Frequently accessed sensor data.
- AI model parameters that are repeatedly used.
- Critical instructions.
- Intermediate processing data.
- Frequently accessed lookup tables.

Because SRAM is extremely fast, it is suitable for **time-critical operations**.

3. DRAM

DRAM (Dynamic Random Access Memory) is a high-capacity volatile memory.

Features

- Higher capacity than SRAM.
- Lower cost per bit.
- Slower than SRAM.

- Requires periodic refreshing.
- Provides good memory density.
- Suitable for storing large amounts of sensor data.

Role in Autonomous Vehicles

DRAM can store:

- Camera frames.
- LiDAR point-cloud data.
- Radar data.
- AI model data.
- Intermediate processing results.
- Operating-system and application data.

DRAM is particularly useful because autonomous vehicles generate **large volumes of sensor data**.

4. MRAM

MRAM (Magnetoresistive Random Access Memory) is a non-volatile memory technology that stores data using magnetic states.

Features

- Non-volatile.
- Fast access.
- Low standby power.
- Good endurance.
- Data is retained even when power is removed.
- Can provide a useful combination of speed and persistence.

Role in Autonomous Vehicles

MRAM can be used for:

- Important configuration information.
- Frequently used parameters.
- System firmware-related data.
- Selected AI model parameters.
- Critical data that should survive power interruptions.

5. Comparison of SRAM, DRAM and MRAM

Feature	SRAM	DRAM	MRAM
Speed	Very high	High	High
Latency	Very low	Higher than SRAM	Low
Volatility	Volatile	Volatile	Non-volatile
Capacity	Low	High	Medium

Feature	SRAM	DRAM	MRAM
Cost/bit	High	Low	Higher than DRAM
Refresh required	No	Yes	No
Standby power	Higher	Moderate	Low
Main application	Cache	Main memory	Fast non-volatile memory

6. Requirements of Autonomous Vehicles

The memory hierarchy must satisfy the following requirements:

Low Latency

Sensor information must be accessed quickly so that the vehicle can respond to obstacles and road conditions.

High Bandwidth

Multiple sensors continuously produce large amounts of data. The memory system must transfer this data quickly.

High Capacity

Camera and LiDAR systems generate large datasets that require substantial memory.

Low Power

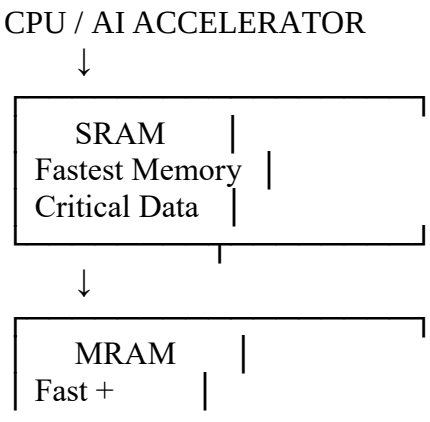
The memory system should consume minimum power to improve overall vehicle efficiency.

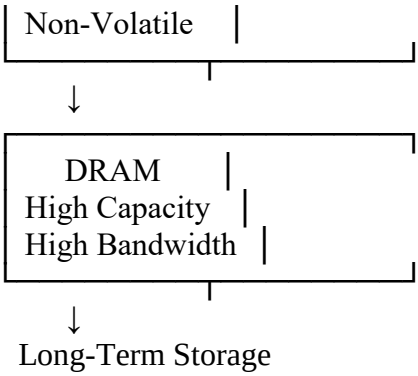
Reliability

Memory should operate reliably under continuous real-time workloads.

7. Proposed Memory Hierarchy

The recommended hierarchy is:





8. Working of the Proposed Hierarchy

Level 1 – SRAM

The most frequently accessed and time-critical data is stored in SRAM.

For example:

Obstacle detection → Critical sensor information → SRAM

This provides extremely fast access.

Level 2 – MRAM

MRAM can store frequently used information that benefits from fast access and non-volatility.

For example:

- AI configuration parameters
- System configuration
- Important persistent information

Level 3 – DRAM

Large volumes of active sensor data are stored in high-bandwidth DRAM.

For example:

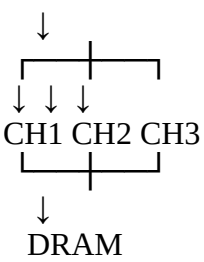
Camera
↓
LiDAR
↓
Radar
↓
Sensor Fusion
↓
DRAM
↓
AI Processing

DRAM provides the capacity needed to handle multiple sensor streams simultaneously.

9. Techniques to Maximize Bandwidth

1. Multiple Memory Channels

Use multiple DRAM channels to transfer several data blocks simultaneously.



This increases overall memory bandwidth.

2. Data Prefetching

Predict the data required next and move it into SRAM before the processor requests it.

3. Parallel Processing

CPU and AI accelerators can process multiple sensor streams simultaneously.

4. Locality

Frequently accessed data should remain in SRAM to minimize slower memory accesses.

5. Efficient Data Transfer

Sensor data can be processed in blocks to reduce unnecessary memory transfers.

10. Techniques to Minimize Latency

1. Store critical data in SRAM.
2. Use efficient cache management.
3. Prefetch frequently required sensor data.
4. Keep frequently used AI parameters close to the processor.
5. Use high-bandwidth DRAM.
6. Reduce unnecessary transfers between memory levels.
7. Use parallel memory channels.
8. Process sensor data locally whenever possible.

11. Example of Real-Time Processing

Suppose a camera detects an obstacle.

Camera
↓
Sensor Data
↓
High-Speed DRAM
↓
Frequently Used Data → SRAM
↓
AI Accelerator
↓
Object Detection
↓
Decision
↓
Brake / Steering

The critical information is kept close to the processor in SRAM so that the AI system can make decisions quickly.

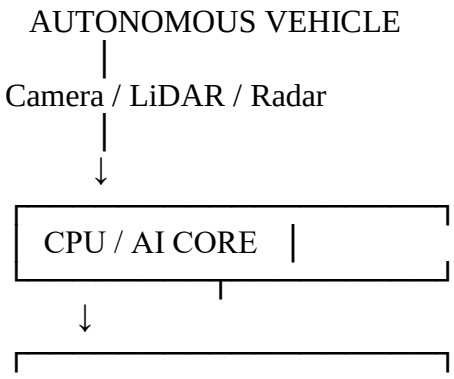
12. Advantages of the Proposed Design

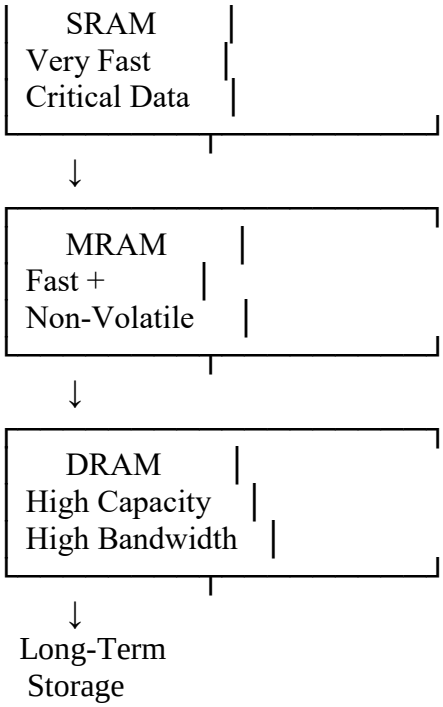
The proposed hierarchy provides:

- **Very low latency** through SRAM.
- **High bandwidth** through DRAM.
- **Non-volatile storage** through MRAM.
- Faster sensor-data processing.
- Better real-time response.
- Efficient AI inference.
- Reduced memory bottlenecks.
- Better energy efficiency.
- Improved reliability.

13. Final Diagram

You can draw this diagram:





SRAM → Minimum latency
MRAM → Fast non-volatile memory
DRAM → High capacity and high bandwidth

Conclusion

For an autonomous vehicle, **SRAM, MRAM and DRAM should be used together rather than as replacements for each other**. SRAM should store the most time-critical data because it provides the lowest latency. MRAM should be used for fast non-volatile information such as configuration and selected parameters. High-bandwidth DRAM should store large sensor datasets and intermediate AI-processing data. This hierarchical design provides **low latency, high bandwidth, sufficient capacity and reliable real-time processing**, making it suitable for autonomous vehicle applications.

4. AI Inference Edge Device

1. Introduction

Edge AI devices such as **smart cameras, drones, robots, IoT devices and wearable devices** perform artificial intelligence processing close to where data is generated. These devices often have limited power and memory resources.

An AI model must be loaded quickly into the processing unit for inference. Therefore, the memory system should provide **fast model loading, low latency, low power consumption and sufficient storage capacity**.

Three important memory technologies are **NAND Flash, DRAM and RRAM**.

2. NAND Flash

NAND Flash is a high-density, non-volatile memory technology.

Features

- Non-volatile.
- Retains data even when power is removed.
- Provides high storage capacity.
- Lower cost per bit.
- Slower than DRAM and RRAM.
- Consumes relatively low standby power.
- Suitable for permanent storage.

Role in Edge AI

NAND Flash can store:

- Complete AI models.
- Operating system.
- Applications.
- Training datasets.
- Images and videos.
- Firmware.

For example, a large AI model can be permanently stored in NAND Flash and loaded when required.

3. DRAM

DRAM (Dynamic Random Access Memory) is a volatile main memory.

Features

- High speed.

- Higher capacity than RRAM.
- Faster than NAND Flash.
- Volatile.
- Requires periodic refresh.
- Higher power consumption than NAND Flash during active operation.
- Suitable for active AI processing.

Role in Edge AI

DRAM stores:

- The currently running AI model.
- Input data.
- Intermediate calculations.
- Feature maps.
- Temporary results.

During AI inference, the model is usually transferred from storage into working memory so the processor can access it quickly.

4. RRAM

RRAM (Resistive Random Access Memory) is a newer non-volatile memory technology.

Features

- Non-volatile.
- Fast read access.
- Low power consumption.
- High density potential.
- Data remains stored without continuous power.
- Can reduce data movement between storage and processing units.

Role in Edge AI

RRAM can be used for:

- Frequently used AI model parameters.
- Model weights.
- Frequently accessed data.
- Fast persistent storage.
- AI accelerators using memory-centric/in-memory processing approaches.

RRAM is particularly attractive for edge AI because it can combine **fast access with non-volatility and low power**.

5. Comparison of NAND Flash, DRAM and RRAM

Feature	NAND Flash	DRAM	RRAM
Volatility	Non-volatile	Volatile	Non-volatile
Speed	Low/Moderate	High	High
Latency	High	Low	Low
Capacity	Very high	High	Medium/High
Power	Low standby	Higher active power	Low
Cost/bit	Low	Moderate	Currently higher
Main use	Permanent storage	Working memory	Fast non-volatile memory

6. Requirements of an Edge AI Device

An optimized memory hierarchy should provide:

Fast Model Loading

The AI model should move quickly from storage to the processing unit.

Low Power Consumption

Edge devices may operate using batteries, so unnecessary memory transfers should be minimized.

Low Latency

AI inference requires rapid access to model parameters and input data.

Sufficient Capacity

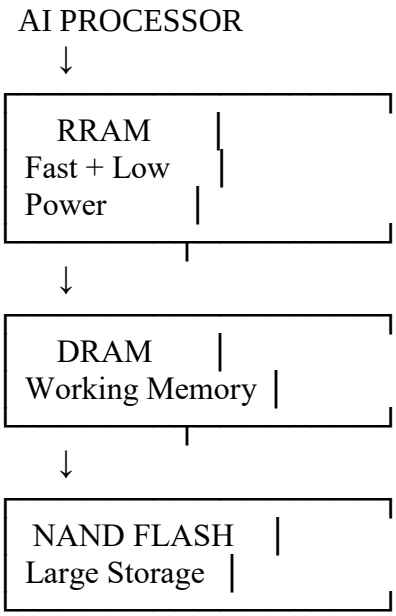
Large AI models require considerable storage.

Efficient Data Movement

Repeatedly transferring model parameters between slow storage and fast memory increases both latency and power consumption.

7. Proposed Memory Hierarchy

The recommended structure is:



8. Working of the Proposed Hierarchy

Level 1 – RRAM

Frequently accessed AI model parameters can be placed in RRAM.

For example:

Frequently Used Model Weights
↓
RRAM
↓
AI Processor

Since RRAM is fast and non-volatile, it can reduce repeated loading of frequently used parameters.

Level 2 – DRAM

DRAM acts as the main working memory.

It stores:

- Active AI model
- Input images
- Sensor data
- Feature maps
- Intermediate results

The AI processor can access this data quickly during inference.

Level 3 – NAND Flash

NAND Flash provides high-capacity permanent storage.

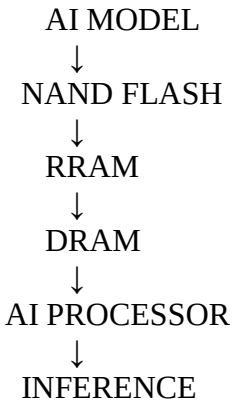
It stores:

- Complete AI models
- Operating system
- Applications
- Large datasets
- Firmware

When a model is required, it can be loaded from NAND Flash into faster memory.

9. Model Loading Process

The model-loading process can be represented as:



Frequently used model parameters can remain available in RRAM, reducing repeated movement from NAND Flash.

10. Methods to Reduce Power Consumption

1. Use RRAM for Frequently Used Data

Keeping frequently accessed parameters in RRAM reduces repeated transfers from NAND Flash.

2. Reduce DRAM Access

DRAM consumes more active power than non-volatile memory. Efficient caching and data reuse can reduce unnecessary DRAM accesses.

3. Model Compression

Techniques such as **quantization and pruning** can reduce the size of AI models.

Smaller models require:

- Less storage.
- Less memory bandwidth.
- Less data movement.
- Less power.

4. Data Locality

Keep frequently used data close to the AI processor.

5. Efficient Memory Management

Only load the required parts of a model rather than unnecessarily moving the entire model.

11. Methods to Improve Model Loading Speed

1. Use high-speed NAND Flash.
2. Use RRAM for frequently accessed model parameters.
3. Provide sufficient DRAM capacity.
4. Use parallel memory access.
5. Compress AI models.
6. Prefetch frequently needed model data.
7. Reduce unnecessary data transfers.
8. Keep frequently used weights closer to the AI processor.

12. Example

Consider an **AI smart camera**.

The camera captures an image and needs to identify a person or vehicle.

Camera
↓
Input Image
↓
DRAM
↓
AI Model Parameters
↓
RRAM
↓
AI Processor
↓
Inference
↓
Object Detected

The complete AI model can remain in NAND Flash, while frequently accessed parameters are kept in RRAM and active processing data is placed in DRAM.

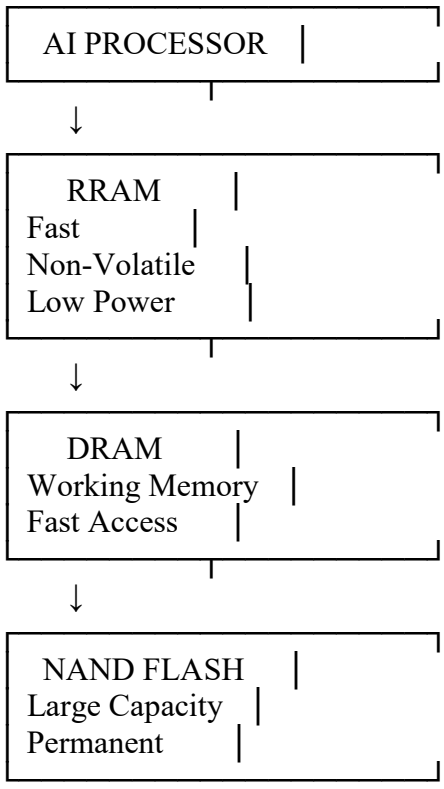
13. Advantages of Proposed Hierarchy

The proposed hierarchy provides:

- Faster AI model loading.
- Lower inference latency.
- Lower power consumption.
- High storage capacity.
- Efficient model-parameter access.
- Reduced data movement.
- Better battery life.
- Improved edge-AI performance.
- Support for larger AI models.

14. Final Diagram for Assessment

Draw this diagram:



↑ Faster / Lower Latency
↓ Larger / Higher Capacity

Conclusion

NAND Flash, DRAM and RRAM have different roles in an edge AI system. **NAND Flash provides high-capacity permanent storage, DRAM provides fast working memory, and RRAM provides fast, low-power non-volatile access.** Therefore, the optimized hierarchy should use **RRAM for frequently accessed model parameters, DRAM for active AI processing, and NAND Flash for large permanent storage**. This structure reduces model-loading latency, minimizes unnecessary data movement and improves the **power efficiency and performance of edge AI inference**.

5. High-Performance Gaming Console

1. Introduction

Modern gaming consoles process large amounts of data while running games. A game scene may contain **high-resolution textures, 3D models, lighting information, audio, animations and physics data**.

When a large scene is loaded, the CPU and GPU require data to be transferred quickly. If the memory system cannot provide data at the required speed, the processor may wait for memory, causing **frame drops, stuttering and reduced gaming performance**.

The two important components in this situation are **L3 cache and DRAM**.

2. L3 Cache

L3 cache is a large cache located close to the CPU and is generally shared by multiple CPU cores.

Features

- Faster than DRAM.
- Larger than L1 and L2 cache.
- Smaller than DRAM.
- Lower latency than DRAM.
- Stores frequently accessed instructions and data.
- Reduces CPU accesses to DRAM.

Role in Gaming

L3 cache can store:

- Frequently accessed game instructions.
- Game-state information.
- Physics calculations.
- Frequently used data.
- AI processing data.

If the required data is available in L3 cache, the CPU can access it much faster than from DRAM.

3. DRAM

DRAM is the main working memory of the gaming console.

Features

- Much larger capacity than L3 cache.
- Higher latency than cache.

- Provides high memory bandwidth.
- Stores large game assets.
- Used by CPU/GPU during game execution.

Role in Gaming

DRAM stores:

- High-resolution textures.
- 3D models.
- Game levels.
- Animation data.
- Audio data.
- Physics data.
- Large buffers used by the GPU.

Large game scenes therefore depend heavily on DRAM capacity and bandwidth.

4. L3 Cache vs DRAM

Feature	L3 Cache	DRAM
Speed	Very high	High
Latency	Very low	Higher
Capacity	Smaller	Much larger
Location	Close to CPU	Main memory
Cost/bit	Higher	Lower
Main purpose	Frequently accessed data	Large working datasets
Best for	Hot/repeated data	Large game assets

5. Why Frame Drops Occur

Frame drops can occur when:

1. Large game scenes require huge amounts of data.
2. The required data is not available in L3 cache.
3. The CPU/GPU must access DRAM frequently.
4. DRAM bandwidth is insufficient.
5. Large textures create heavy memory traffic.
6. Data is transferred inefficiently.
7. Cache misses increase.
8. Assets are loaded too late.

The CPU or GPU may remain idle while waiting for required data.

6. L3 Cache Throughput vs DRAM Throughput

L3 Cache

L3 cache provides **low-latency access** and is useful for repeatedly accessed data.

CPU
↓
L3 Cache
↓
Frequently Used Data
↓
Fast Processing

However, L3 cache has limited capacity, so it cannot store an entire large game scene.

DRAM

DRAM provides much greater capacity and high bandwidth.

CPU / GPU
↓
DRAM
↓
Large Game Assets
↓
Textures / Models / Audio

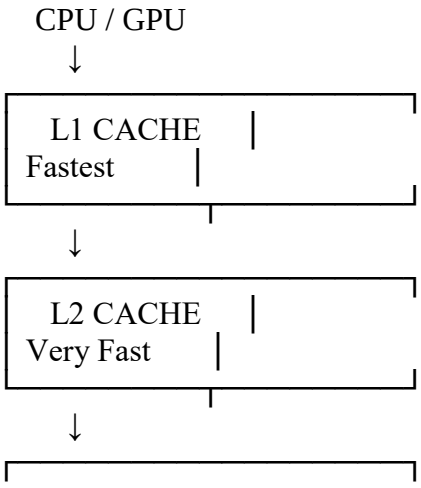
However, DRAM has higher latency than cache.

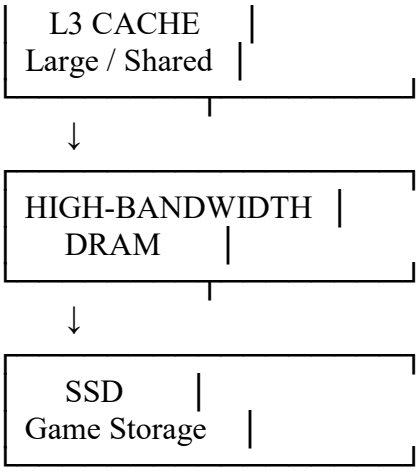
Therefore:

L3 cache minimizes latency, while DRAM provides the capacity and bandwidth needed for large game datasets.

7. Proposed Memory Hierarchy

The recommended gaming-console hierarchy is:





8. Proposed Enhancements

1. Increase L3 Cache Capacity

Increasing L3 cache allows more frequently accessed game data to remain close to the CPU.

Benefit:

- Fewer cache misses.
- Lower CPU memory latency.
- Smoother game execution.

2. Use High-Bandwidth DRAM

Large scenes require continuous transfer of textures and other assets.

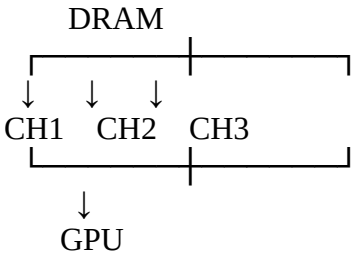
Using high-bandwidth memory such as **GDDR-class memory** can improve data-transfer performance.

Benefit:

- Faster texture loading.
- Faster GPU data access.
- Better frame stability.

3. Increase Memory Channels

Multiple memory channels allow data to be transferred in parallel.



This increases the effective memory bandwidth.

4. Use Prefetching

The system can predict which assets will be required next and load them before they are needed.

For example:

Current Scene
↓
Predict Next Scene
↓
Prefetch Assets
↓
DRAM
↓
Ready for GPU

This reduces waiting time during scene transitions.

5. Improve Cache Management

Frequently accessed game data should remain in L3 cache.

Efficient cache replacement policies can prevent useful data from being removed unnecessarily.

6. Asset Compression

Large textures and game assets can be compressed.

This reduces:

- Memory usage.
- DRAM traffic.
- Storage requirements.
- Data-transfer time.

7. Fast SSD Storage

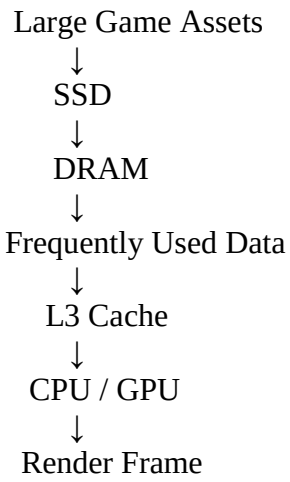
A high-speed SSD can provide faster loading of large game assets.

The SSD should supply data to DRAM efficiently during scene loading.

Example of Large Scene Loading

Suppose the player moves from one game area to another.

New Game Scene
↓



If assets are loaded too slowly from storage to DRAM, or if DRAM cannot provide data quickly enough to the GPU, frame drops may occur.

How the Proposed Design Reduces Frame Drops

Before Improvement

Large Scene
↓
SSD
↓
DRAM
↓
DRAM Bandwidth Bottleneck
↓
CPU/GPU waits
↓
Frame Drop

After Improvement

Large Scene
↓
Fast SSD
↓
High-Bandwidth DRAM
↓
Prefetching
↓
L3 Cache for Hot Data
↓
CPU/GPU
↓
Smooth Rendering

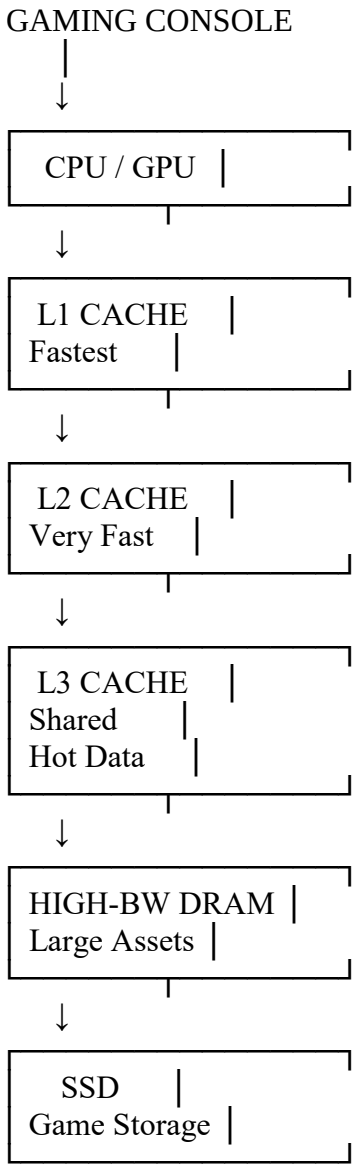
Expected Benefits

The proposed memory hierarchy provides:

- Reduced frame drops.
- Reduced game stuttering.
- Faster scene loading.
- Higher memory bandwidth.
- Lower CPU memory latency.
- Better GPU utilization.
- Faster texture access.
- Improved overall gaming performance.
- Smoother frame rates.

12. Final Diagram

Draw this diagram:



Cache → Low Latency

DRAM → High Bandwidth
SSD → Large Storage

Conclusion

L3 cache and DRAM have different but complementary roles in a gaming console. **L3 cache provides low-latency access to frequently used game data, while DRAM provides the high capacity and bandwidth required for large textures, models and other game assets.** To prevent frame drops, the console should use a larger and efficient L3 cache, high-bandwidth DRAM, multiple memory channels, asset prefetching, compression and fast SSD storage. This optimized hierarchy reduces memory bottlenecks and provides **faster scene loading, higher throughput and smoother gaming performance**.